

University of Trento

Optimisation and Learning for Robot Control

~ · ~

February 6, 2026

Final Project Report
Project A

Professor and tutor:
Andrea Del Prete
Elisa Alboni

Students:
Alessandro Rossi 256240
Lorenzo Franco 257585

Contents

1 Introduction **1**

2 Dataset Generation **2**

3 Neural Network Training **5**

4 MPCs Comparison **8**

5 Conclusion **13**

1 Introduction

The following report contains the work on the final assignment of the students Alessandro Rossi 256240 and Lorenzo Franco 257585.
The focus of the project A is to learn an approximate value function that can be then used as terminal cost in an MPC formulation. To achieve this, students are required to train their own neural network.
Contrary to the previous homework, where the assignment was structured as a tutorial (by guiding the students step by step through a pre-written code), the final project requires implementing the neural network almost from scratch.
As suggested in the assignment description, the project has been developed previously for a single pendulum system. After completion and testing, the code has been adapted to work with a double pendulum system. The single pendulum will not be discussed in the following, as it is qualitatively similar to the double pendulum case. The following report is structured in the three main phases of the project development: Dataset Generation, Neural Network Training and MPCs Comparison. A final conclusion section summarizes the results obtained and suggests possible future developments.

2 Dataset Generation

For the first part of the project, the dataset, subsequently used to train the neural network and to compute the OCP final cost, is to be generated. Therefore, the OCP problem for the double pendulum has to be defined. The double pendulum system consists of two links connected in series, with the first link attached to a fixed pivot point. The system is under-actuated, meaning that only one control input (torque) is applied at the first joint, while the second joint is passive. The goal of the OCP is to drive the system from a given initial state to the upright equilibrium position while minimizing a cost function.

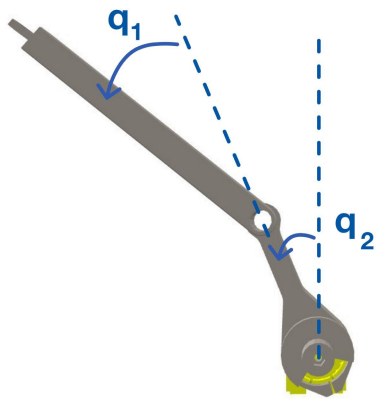


Figure 2.1: Double pendulum coordinates.

Using the notation in Figure 2.1, the state of the system was defined as $\mathbf{x} = [q_1, q_2, \dot{q_1}, \dot{q_2}]^T$. Where q_1 and q_2 are the angles, while $\dot{q_1}$ and $\dot{q_2}$ are the angular velocities of the first and second links, respectively. The control input was defined as the torque $\mathbf{u} = \tau_1$ applied to the first joint, at the second joint, as previously mentioned, no torque is applied ($\tau_2 = 0$).

The OCP problem was defined as follows:

$$\begin{aligned} \min_{\mathbf{X},\mathbf{U}} \quad & \sum_{k=0}^{N-1} \Big((\mathbf{x}_k - \mathbf{x}_{targ})^\top \mathbf{W}_x (\mathbf{x}_k - \mathbf{x}_{targ}) + (u_k - u_{targ})^\top \mathbf{W}_u (u_k - u_{targ}) \Big) \\ \text{subject to} \quad & \mathbf{x}_0 = \mathbf{x}_{init} \\ & \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, u_k), \quad k = 0, \ldots, N-1 \end{aligned} \tag{2.1}$$

Where:

- $\mathbf{x}_{targ} = [0, 0, 0, 0]^\top$ is the target state.
- $\mathbf{u}_{targ} = [0]$ is the reference control input at equilibrium.
- $\mathbf{W}_x = \text{diag}(4e^0, 4e^0, 5e^{-1}, 1e^{-2})$ is the state weight matrix.
- $\mathbf{W}_u = [1e^0]$ is the control weight matrix.
- $\mathbf{x}_{init}$ is the initial state, which varies for each value in the dataset.
- $\mathbf{f}(\mathbf{x}_k, u_k)$ represents the system dynamics.

Thus, the OCP cost was defined, within the CasADi framework, as a quadratic cost, penalizing the distance from the upright position, velocities, and the use of control input. The weights are chosen to allow the solver to find a solution in most initial conditions and to obtain physically reasonable motions.

It was later found that the cost defined in (2.1) was not the most suitable for this problem, but this aspect will be discussed in the next sections.

The system's dynamics were modeled using the ADAM (`from adam.casadi`) library, which allows to compute the dynamics (thus the mass matrix $M(\mathbf{q})$ and bias forces $h(\mathbf{q}, \dot{\mathbf{q}})$) directly from the robot's URDF file. ($\mathbf{q} = [q_1, q_2]^\top$ are the joint angles vector).

This permits to solve the manipulator equation and find the joint accelerations $\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}(\boldsymbol{\tau} - \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}))$, where $\boldsymbol{\tau} = [\tau_1, 0]^\top$ accounts for the system's under-actuation. These accelerations were concatenated with the joint velocities $\dot{\mathbf{q}}$ to form the state derivative $\dot{\mathbf{x}} = [\dot{q_1}, \dot{q_2}, \ddot{q_1}, \ddot{q_2}]^\top$. Finally, the discrete-time state transition is achieved via an explicit Euler integration scheme, defined as $\mathbf{x}_{k+1} = \mathbf{x}_k + \dot{\mathbf{x}}_k \Delta t$, which maps the current state to the next over the time step Δt. So the function $\mathbf{f}(\mathbf{x}_k, u_k)$ in (2.1) was defined as:

$$\mathbf{f}(\mathbf{x}_k, u_k) = \mathbf{x}_k + \dot{\mathbf{x}}_k \Delta t \ = \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix}_k + \begin{bmatrix} \dot{\mathbf{q}} \\ \ddot{\mathbf{q}} \end{bmatrix}_k \cdot \Delta t \tag{2.2}$$

The following parameters were chosen for the OCP solver:

- Number of discretization steps: $N = 100$ steps
- Time step: $\Delta t = 0.05$ s

With all of this set up, the OCP problem in (2.1) was solved 10.000 times, each with a different initial condition $\mathbf{x}_{init}$. At every run the initial state and the obtained cost were saved to generate the dataset. The initial conditions were sampled randomly from a uniform distribution within the following ranges:

- Joint 1 position (q_1): sampled from a uniform distribution in the range $[-\pi, \pi]$ rad.
- Joint 2 position (q_2): sampled from a uniform distribution in the range $[-0.9\pi, 0.9\pi]$ rad.
- Joint 1 velocity ($\dot{q}_1$): sampled from a uniform distribution in the range $[-8.0, 8.0]$ rad/s.
- Joint 2 velocity ($\dot{q}_2$): sampled from a uniform distribution in the range $[-15.0, 15.0]$ rad/s.

The ranges were selected to cover a broad spectrum of initial conditions, ensuring that the dataset captures a diversity of trajectories necessary to training the neural network. Simultaneously, the boundaries are set not to be too broad, in order to ensure that the state space remains sufficiently explored, maintaining a high density of samples within the region of interest to facilitate effective learning.

The samples of q_2 are limited to a value of 90% of π to avoid singularities in the dynamics computation when the second link is fully inverted.

The multiprocessing library of Python was also used, to parallelize the OCP solving process, speeding up the dataset generation.
An example of the motion obtained by solving the OCP from different initial conditions is shown in the video at the following link: `https://youtu.be/N8cDjysMnIY`.
The initial conditions were:

- Example 1: $\mathbf{x}_{init} = [2.88, 0.89, -10.0, -10.0]^\top$
- Example 2: $\mathbf{x}_{init} = [2.88, -0.89, 0.0, 0.0]^\top$
- Example 3: $\mathbf{x}_{init} = [0.47, -0.47, 0.0, 1.5]^\top$
- Example 4: $\mathbf{x}_{init} = [-1.2, 0.14, 1.65, 0.53]^\top$

Finally, in the following Figure 2.2, the dataset generation is reported:

```
ale_rossi@LAPTOP-E63HJQH0:~/delprete/Assignments$ /bin/python3 /home/ale_rossi/delprete/Assignments/Git/Optimum_learn_Robot_Control/Final_Proj/Do
uble_Pendulum/fase1_dataset_from_OCP_doub_pend.py
 [!] Solver Failed for x_init: [ -2.91   2.24  -1.85 -13.87]
 [!] Solver Failed for x_init: [ -2.33   2.27   0.25 -12.44]
 [!] Solver Failed for x_init: [ 1.88 -2.07  6.68 14.01]
 [!] Solver Failed for x_init: [ 1.88 -2.45  3.45 12.87]
 [!] Solver Failed for x_init: [ 2.31 -2.32 -1.47 13.58]
 [!] Solver Failed for x_init: [ 2.68 -2.27  3.46  8.25]
 [!] Solver Failed for x_init: [ -2.59   2.29   1.9  -14.78]
 [!] Solver Failed for x_init: [ 2.73 -2.69  6.71 11.61]
 [!] Solver Failed for x_init: [ -2.64   2.63  -2.92 -11.34]
 [!] Solver Failed for x_init: [ 2.62 -2.4   5.05 13.27]
 [!] Solver Failed for x_init: [ 3.01 -2.57  2.8   6.65]
 [!] Solver Failed for x_init: [-2.69  2.31 -0.35 -6.77]
Solving OCPs: 100%|                                        | 10000/10000 [33:21<00:00,  5.00it/s, Costs=767.48, Fails=58, Succ. Rate=99.42%]

Dataset created. Final Success Rate: 99.42%
```

Figure 2.2: Dataset generation results.

As can be seen, the solver has a success rate of 99.42%, meaning that 9942 trajectories were successfully computed out of the 10000 initial conditions sampled. The time to perform the whole dataset generation was 32.21 minutes using 8 parallel processes.
Afterwards, the dataset is saved as a .npz file called `pendulum_dataset_used.npz`, containing two arrays: `inputs` of shape $(9942, 4)$ reporting the initial states and `targets` of shape $(9942, 1)$ illustrating the corresponding costs. The dimension is not 10000 because some trajectories failed to be computed by the solver.

All of this was implemented in the file: "`fase1_dataset_from_OCP_doub_pend.py`"

The content of the dataset was then visualized using a Parallel Coordinates Plot, shown in Figure 2.3:

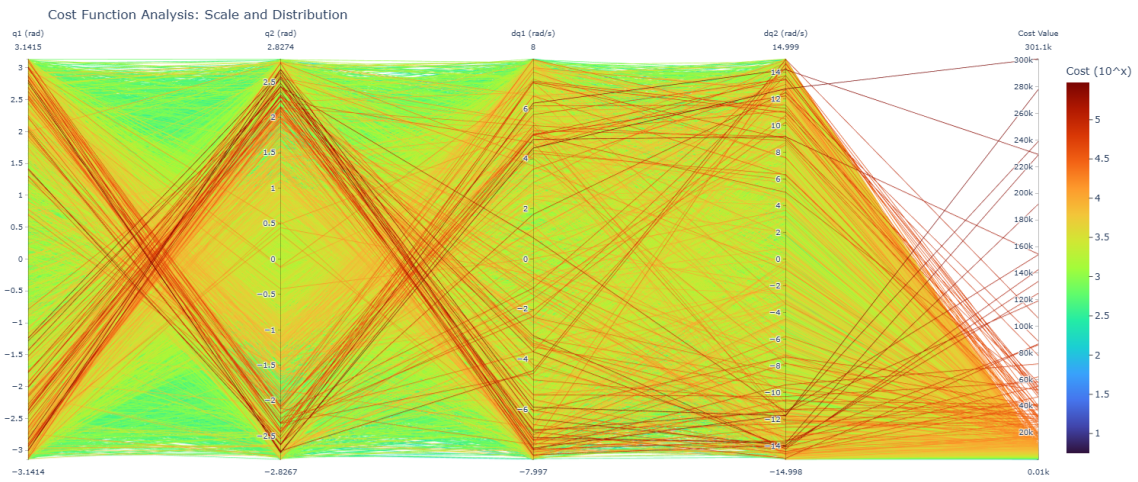


Figure 2.3: Parallel Coordinates Plot of the dataset.

From the plot, it can be observed that the cost values span several orders of magnitude, ranging from approximately 10^0 to 10^5. This wide range indicates that some initial conditions lead to significantly higher costs than others. However the vast majority of the values are under 20 000. For this reason we considered the values above this number as outliers and we decided to remove them from the dataset before training the neural network. This choice was made to prevent these extreme values from skewing the training process and adversely affecting the model's performance on more typical scenarios. (To interact with the plot, please run the `dataset_cost_plot.py` file).

3 Neural Network Training

Once the dataset generation has been completed, it can be used to train a neural network able to predict the final OCP cost. The data contained in the `.npz` file, although it has been prepared to be fed to the neural network, some preprocessing steps are required.
The first consists of removing the outliers present in the dataset. This was done by setting a threshold on the cost values and removing all samples with a cost higher than 20000. These samples were considered too far from the average cost value and would have negatively affected the training process.
The second step consisted in bounding both the inputs (denoted by $\mathbf{X}$) and the targets (denoted by $\mathbf{Y}$) to the range $[-1, 1]$. This choice was motivated by the use of the `tanh` activation function in the neural network, whose output lies in the same interval. Moreover, neural networks generally perform better when the input data is normalized.
Additionally the targets, before scaling, were transformed using a logarithmic scale according to:

$$\mathbf{Y}_{\log} = \ln(\mathbf{Y} + 10^{-8}) \tag{3.1}$$

This transformation was introduced because the cost values spanned several orders of magnitude. Using a logarithmic scale helped to narrow the spread of the targets, making the training process more stable. The small constant 10^{-8} was added to avoid taking the logarithm of zero. Additionally, during preliminary tests, the network showed poor accuracy when predicting cost values close to zero. The logarithmic transformation significantly improved the prediction accuracy in this region.
When using the trained network to predict the final cost, the inverse transformation was applied in order to recover the actual cost values.
The dataset was then split into training and test sets, using 80% of the samples for training and 20% for testing. This split allowed us to monitor the performance of the network on unseen data during training and to reduce the risk of overfitting.
The neural network architecture was defined as follows:

- One input layer with 4 neurons, corresponding to the state variables (position and velocity);
- Two hidden layers with 256 neurons each;
- One output layer with a single neuron representing the predicted cost;
- The `tanh` activation function was used for all layers.

The training process was implemented using batch gradient descent. For each training epoch, batches of $n = 256$ samples were drawn from the training set. For each batch, the network performed a forward pass, mapping the initial pendulum state to a predicted cost, followed by the computation of the Mean Squared Error (MSE), defined as:

$$\text{MSE}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} \left(\phi(\mathbf{x}_i, \mathbf{w}) - y_i\right)^2 \tag{3.2}$$

Where $\mathbf{w}$ denotes the network weights, $\phi(\mathbf{x}_i, \mathbf{w})$ is the predicted cost associated with the input state $\mathbf{x}_i$, and y_i is the corresponding true cost value from the dataset.
For each batch within an epoch, the weights $\mathbf{w}$ were updated by applying the gradient descent to the MSE loss, this was implemented via the optimization algorithm Adam (Adaptive Moment Estimation) from the library PyTorch (`torch.optim.Adam`). It is important to distinguish this optimization algorithm from the ADAM robotics library used in Chapter 2 for rigid body dynamics; while they share the same name, the former is an algorithm for training neural

networks, whereas the latter is a software framework for kinematic and dynamic computations. The learning rate was initially set to 10^{-3} and progressively decayed down to a minimum value of 10^{-5} using a learning rate scheduler. In particular, PyTorch's `ReduceLROnPlateau` scheduler was employed, which reduces the learning rate when the test loss (defined in (3.4)) does not improve for a specified number of epochs. This strategy helped stabilize the training process and slightly improved the final performance.
At the end of each epoch, the training loss was computed as the average of the MSE values over all training batches:

$$\text{Loss}_{\text{train}}(\mathbf{w}) = \frac{1}{m}\sum_{j=1}^{m}\text{MSE}_j(\mathbf{w}) \tag{3.3}$$

Where m denotes the total number of batches in the training set. This metric was used to monitor the learning progress.
The testing procedure followed the same structure as the training process, with the only difference being that the network weights were not updated. The test loss was computed as

$$\text{Loss}_{\text{test}}(\mathbf{w}) = \frac{1}{m}\sum_{j=1}^{m}\text{MSE}_j(\mathbf{w}) \tag{3.4}$$

Where m is the number of batches in the test set.
The entire training process was repeated for a total of 600 epochs. Both training and testing losses were monitored throughout the training, and the resulting trends are shown in Figure 3.1.

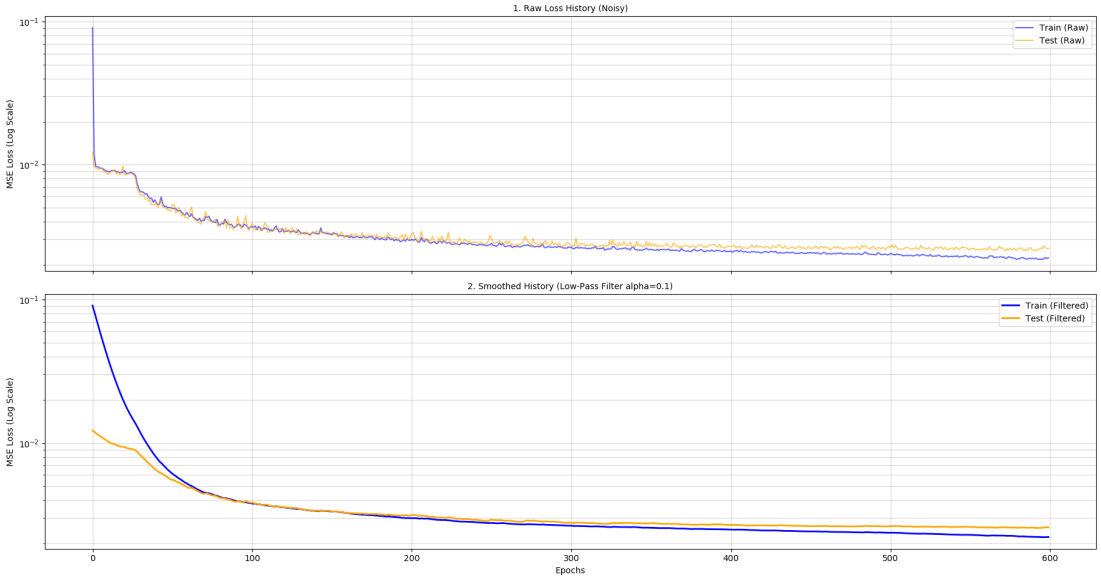


Figure 3.1: Training and testing losses over the training epochs.

As shown in the figure, a low-pass filter was applied to the loss curves to better visualize their overall trends. The training loss decreases steadily over the epochs, indicating that the network effectively learns from the training data. The testing loss follows a similar decreasing trend, suggesting good generalization performance and the absence of significant overfitting.
An example of the code used during the training process is reported in Figure 3.2.

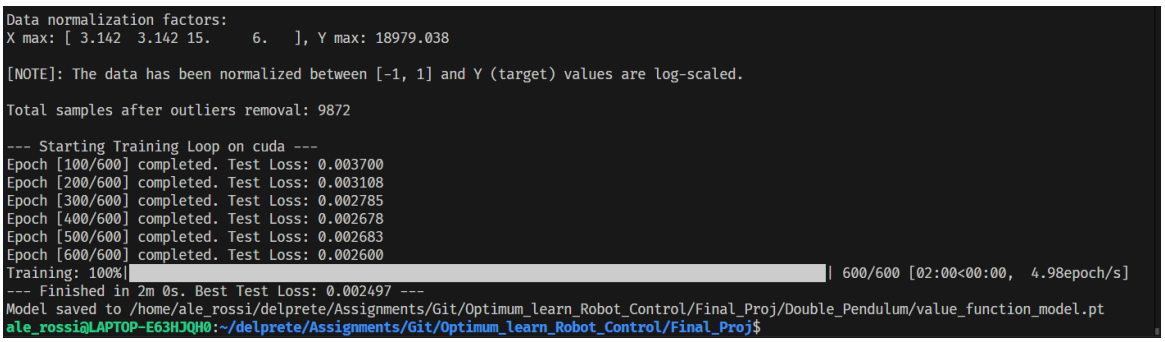


Figure 3.2: Example of code used for the training process.

The overall training procedure required approximately two minutes to complete, thanks to the use of GPU acceleration available on one of the laptops.

All of this was implemented in the file: "`fase2_network_training_doub_pend.py`"

4 MPCs Comparison

The final step requires to add a final cost term to the OCP in (2.1), enabling the computation using the trained neural network. Thus the new OCP becomes:

$$\begin{aligned} \min_{\mathbf{X},\mathbf{U}} \quad & \sum_{k=0}^{M-1} \Big((\mathbf{x}_k - \mathbf{x}_{targ})^\top \mathbf{W}_x (\mathbf{x}_k - \mathbf{x}_{targ}) + (u_k - u_{targ})^\top \mathbf{W}_u (u_k - u_{targ}) \Big) + J(\mathbf{x}_M) \\ \text{subject to} \quad & \mathbf{x}_0 = \mathbf{x}_{init} \\ & \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, u_k), \quad k = 0, \dots, M-1 \end{aligned} \tag{4.1}$$

Where $J(\mathbf{x}_M)$ is the final cost computed using the trained neural network, given the final state $\mathbf{x}_M$ at the end of the horizon.

The objective of this part is to compare three different MPC controllers:

- Without final cost and horizon M < N
- With final cost computed using the network cost function and horizon M < N
- Without final cost and horizon M + N

Where N is the full horizon length used during the dataset generation.
It is to be expected to see the last two controllers perform similarly, while the first one would be less effective, since it does not account for the final cost and has a shorter horizon.

The first operation is to create a CasADi function that implements the neural network trained in the previous section. This function takes into account the normalization steps used during training. So it takes as input the state $\mathbf{x}$, normalizes it $\mathbf{x}_{norm}$ and outputs the estimated cost $J(\mathbf{x}_{norm})$ by re-converting it back from logarithmic scale.
To speed up the MPC computation and obtain better results, a warm-starting strategy has been implemented. At each MPC iteration, the previous solution is used as an initial guess for the next optimization problem. Specifically, the state and control trajectories from the previous solution are shifted forward by one time step, and the last state is repeated to fill in the gap created due to the shift. This approach helped providing a good initial guess for the solver and showed better convergence speed and solution quality.
The values chosen for the MPC controllers are:

- Short-Horizon length: $M = 14$
- Long-Horizon length: $M + N = 114$

Obviously, all the hyperparameters used in the OCP definition are the same as in (2.1).

With all of this set up, the three MPC controllers are tested on 30 randomly sampled initial conditions:

- Joint 1 position (q_1): uniformly sampled in $[-\pi, \pi]$
- Joint 2 position (q_2): uniformly sampled in $[-0.8\pi, 0.8\pi]$
- Angular velocities ($\dot{q}_1, \dot{q}_2$): uniformly sampled in $[-1.0, 1.0]$ rad/s

Two examples of the trajectories obtained with the three different MPC controllers are shown in Figure 4.1 and Figure 4.2, by using the same initial condition:

- Initial Conditions 1: $\mathbf{x}_{init} = [-0.820,\ -0.60,\ 0.990,\ -0.830]^\top$
- Initial Condition 2: $\mathbf{x}_{init} = [-0.237,\ 0.612,\ 0.732,\ 0.596]^\top$

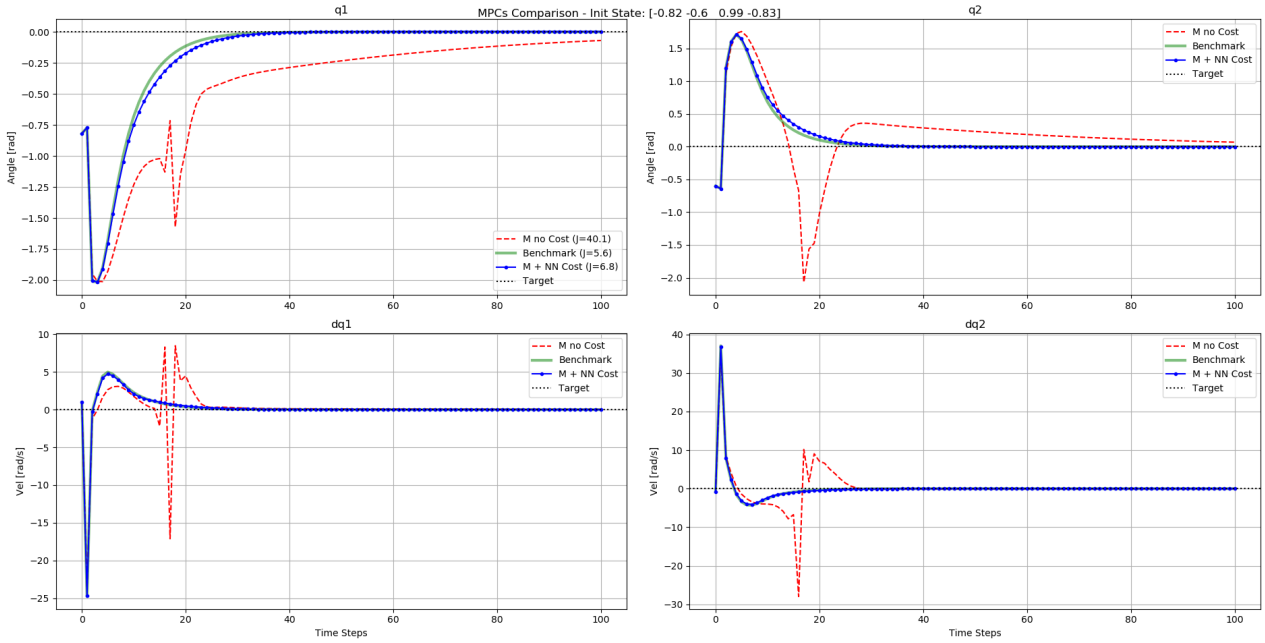


Figure 4.1: Trajectories obtained with the three different MPC controllers using initial conditions 1.

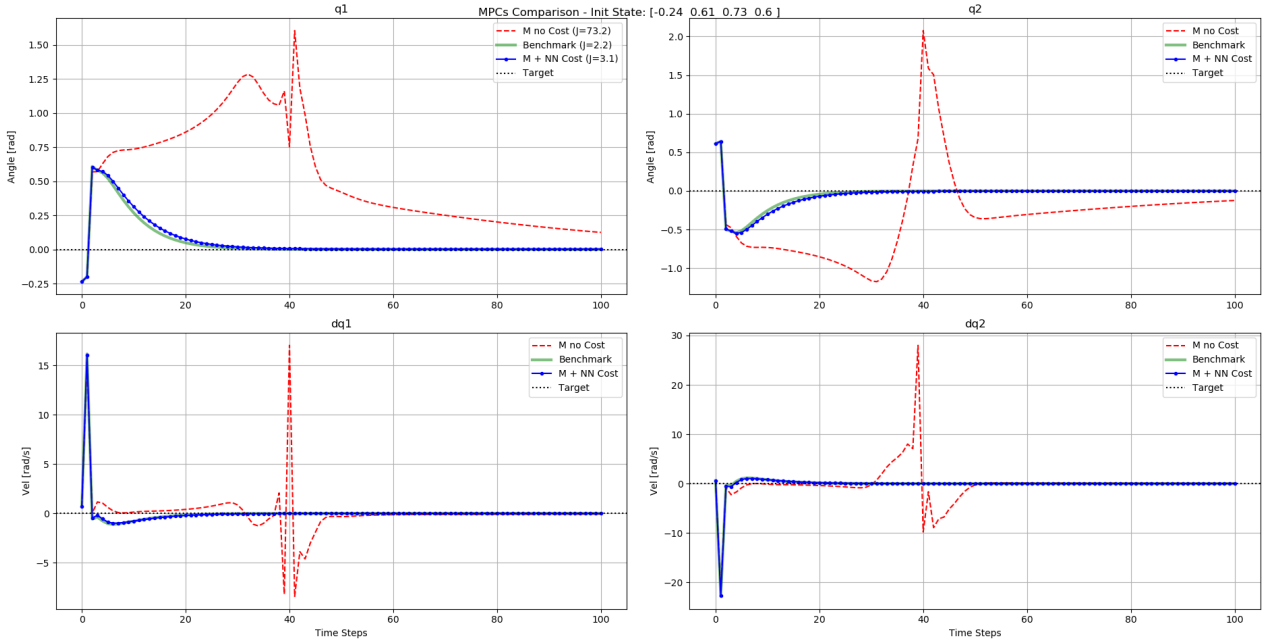


Figure 4.2: Trajectories obtained with the three different MPC controllers using initial condition 2.

As can be seen, in both cases the controller with final cost (blue) and the one with the long horizon (green) perform similarly, while the one without final cost and the long horizon (red) performs worse, as expected.

By following the links, the videos of the trajectories above can be seen:

- Initial Conditions 1: `https://youtu.be/dTzMtQ39pxc`
- Initial Conditions 2: `https://youtu.be/vgQT7q9oPIk`

Some multiple run trajectories are reported in: `https://youtu.be/99hOP7C9648`. The initial states of these examples are:

- $\mathbf{x}_{init} = [\pi, 0.0, 0.0, 0.0]^\top$
- $\mathbf{x}_{init} = [1.1415, 2.827, 0.5, -0.9999]^\top$
- $\mathbf{x}_{init} = [-2, -0.60, 0.3, -0.83]^\top$
- $\mathbf{x}_{init} = [0.1, -2.60, 0.6, -0.23]^\top$

To compare the three controllers quantitatively, the total cost is computed over the 30 runs. However, the cost defined in (4.1) turned out to be not very informative, since some times the trajectories obtained were clearly better in the case of the MPC with final cost or long horizon, but their total cost was higher with respect to the short horizon case. This is due to the fact that the cases with final cost or long horizon tend to reach the target state more aggressively. This implies high values of the state and control errors during the initial part of the trajectory. This difference value is then squared, making the total cost higher. On the other hand the case with short horizon tends to reach the target in more time, resulting in lower errors peaks and thus a lower total cost.

For this reason, the comparison has been held on a different cost, based on metrics. This is not an optimal solution, the OCP cost should have been redefined, but given the time constraints, it has been set to proceed this way. The results obtained in this manner are still meaningful and of interest.

The new cost has been computed from the one in (4.1) by applying the square root to the state and control error terms, thus obtaining a cost that penalizes linearly the errors instead of quadratically. A weight w_t which is increased linearly with time has also been implemented, to further penalize the errors at the end of the trajectory. Thus the new cost becomes:

$$L(\mathbf{x}_k, \mathbf{u}_k) = \sqrt{(\mathbf{x}_k - \mathbf{x}_{ref})^T W_x (\mathbf{x}_k - \mathbf{x}_{ref})} + \sqrt{(\mathbf{u}_k - \mathbf{u}_{ref})^T W_u (\mathbf{u}_k - \mathbf{u}_{ref})} \tag{4.2}$$

$$w_t = \left(\frac{k+1}{N}\right) \tag{4.3}$$

$$J_{comparison} = \sum_{k=0}^{N-1} w_t \cdot L(\mathbf{x}_k, \mathbf{u}_k) \tag{4.4}$$

Where:

- $L(\mathbf{x}_k, \mathbf{u}_k)$ is the instantaneous cost at time step k
- $J_{comparison}$ is the total cost used for comparison
- N is the total number of time steps in the trajectory

A second metric used for the comparison, is the time taken by each pendulum to reach the target state within a certain state threshold. This was done by monitoring if the Euclidean

distance between the current state $\mathbf{x}_k$ and the target equilibrium $\mathbf{x}_{ref}$ was below an arbitrary threshold of 0.08.

The results obtained over the 30 simulations are reported in Table 4.1.
As can be seen 25 times out of 30, the MPC with final cost performed better than the short horizon one without final cost. This means that 83% of the times the use of the learned final cost improved the performance. In the other 5 cases, the MPC with the final cost performed significantly worse, this is probably due to some limitations in the neural network that can be probably be solved by better training.

By analyzing the time taken to reach the target state, the MPC with short horizon and without final cost was essentially unable to reach the target within the treshold in most of the cases, while the MPC with final cost reached the target in 27 cases out of 30 (90% of success rate). This shows that the use of the learned final cost significantly improves the controller's ability to reach the desired state within a reasonable time frame.

NOTE: when the time is -1 it means that the controller was unable to reach the target within the distance threshold.

Table 4.1: Comparative performance analysis of MPC strategies across 30 random initial states. Case 1: Short horizon ($M = 14$) without terminal cost. Case 2: Short horizon ($M = 14$) with Learned NN Terminal Cost. Case 3: Long horizon benchmark ($N + M = 114$). Negative time values (-1.00) indicate a failure to converge within the distance threshold.

Run	**Initial State**	**Case 1 (M=14)**		**Case 2 (M+NN)**		**Case 3 (N+M=114)**	
	$[q_1, q_2, \dot{q}_1, \dot{q}_2]$						
#	[rad, rad, rad/s, rad/s]	Cost	Time [s]	Cost	Time [s]	Cost	Time [s]
1	[1.53, 1.32, 0.28, 0.58]	23.846	4.95	16.00	2.10	12.607	1.75
2	[0.12, 0.95, 0.18, 0.40]	57.763	-1.00	4.24	1.55	3.191	1.40
3	[-2.68, 0.69, 0.99, -0.00]	36.308	-1.00	11.75	1.85	10.661	1.65
4	[-0.56, -0.75, -0.31, -0.50]	39.765	-1.00	6.13	1.70	5.086	1.50
5	[-1.27, -1.59, -0.82, 0.90]	20.865	4.65	3.84	1.45	3.264	1.35
6	[-2.82, 2.32, -0.28, 0.07]	33.267	-1.00	13.25	1.95	8.315	1.65
7	[1.36, 0.62, 0.19, -0.08]	24.683	-1.00	9.52	1.90	7.565	1.60
8	[0.81, 0.56, -0.80, 0.41]	37.060	-1.00	7.23	1.75	5.725	1.55
9	[0.41, 1.61, -0.72, -0.05]	15.513	4.05	2.93	1.25	2.238	1.15
10	[-0.76, -1.87, 0.71, -0.94]	34.390	-1.00	83.85	4.05	2.533	1.25
11	[2.03, -0.72, -0.69, -0.98]	28.864	-1.00	10.52	1.85	9.161	1.60
12	[1.35, -0.84, 0.54, -0.85]	28.764	-1.00	157.57	-1.00	7.811	1.60
13	[2.75, -0.41, 0.64, 0.24]	34.007	-1.00	65.18	-1.00	11.137	1.70
14	[1.83, -1.24, 0.05, 0.75]	31.062	-1.00	8.96	2.00	6.909	1.60
15	[1.74, 1.24, 0.65, -0.38]	25.747	-1.00	4.02	1.45	3.936	1.40
16	[-2.08, -0.01, -0.25, 0.41]	29.259	-1.00	10.25	1.80	9.219	1.65
17	[-2.21, -1.59, 0.40, -0.45]	26.836	-1.00	5.84	1.60	5.615	1.50
18	[-1.44, -1.17, 0.60, 0.61]	34.476	-1.00	7.17	1.75	6.043	1.60
19	[-2.89, 1.94, -0.09, -0.69]	36.587	-1.00	15.19	2.35	32.760	2.05
20	[2.43, -0.54, 0.04, -0.34]	36.559	-1.00	15.52	1.95	10.134	1.65
21	[2.07, -1.98, 0.66, -0.82]	34.207	-1.00	216.30	-1.00	6.180	1.55
22	[-1.41, -0.37, -0.60, -0.10]	25.576	-1.00	9.33	1.85	7.981	1.60
23	[-0.71, 1.55, -0.98, 0.62]	5.506	1.90	1.41	0.60	0.859	0.60
24	[2.90, 2.04, 0.18, 0.92]	30.096	-1.00	7.16	1.65	6.141	1.55
25	[1.67, 1.84, -0.95, 0.89]	19.431	4.50	4.09	1.45	3.271	1.35
26	[-0.06, 1.59, 0.09, -0.28]	12.398	3.60	2.44	1.15	1.784	1.05
27	[2.48, 0.74, -0.15, 0.26]	37.187	-1.00	53.89	3.15	9.047	1.65
28	[-2.71, 0.05, 0.01, 0.74]	31.776	-1.00	11.33	1.85	10.126	1.65
29	[-0.51, -0.83, -0.41, -0.65]	44.524	-1.00	5.79	1.65	4.802	1.50
30	[2.55, 0.69, 0.14, -0.14]	33.484	-1.00	14.49	1.95	9.994	1.70

5 Conclusion

In the end the results obtained were acceptable, even if not excellent. The MPC developed showed to be effective in making the pendulum reach the desired configuration, but the performance could surely be improved.
We believe that the main source of improvement could be the redefinition of the cost function in (2.1). As currently defined, it outputs values across several orders of magnitude, which likely makes the neural network training more difficult. Also as mentioned in the previous section, the cost (2.1) sometimes was lower with the MPC with short horizon and no final cost, even if the performance was visibly worse.

Despite these observations, we decided to not change the cost function, because it would have required more time to define and tune a new cost to obtain good results.
Moreover, the current results already provide a clear validation of the approach's potential, making further refinement of the cost function unnecessary for this proof of concept.